

Sounio: Epistemic Types for Scientific Computing with Native Uncertainty Quantification

Demetrios Chiuratto Agourakis
demetrios@sounio-lang.org
<https://souniolang.org>

February 2026

Abstract

Scientific computing requires rigorous handling of measurement uncertainty, yet systems programming languages lack native support for propagating uncertainties in compliance with the Guide to the Expression of Uncertainty in Measurement (GUM, JCGM 100:2008). This leads to error-prone manual implementations and challenges in regulatory contexts such as FDA 21 CFR Part 11 and ISO 17025.

We introduce **Sounio**, a systems programming language for *epistemic computing*—computation where uncertainty, confidence, and provenance are first-class type-level concerns. The `Knowledge<T>` type encapsulates a value, its GUM-compliant variance, a Beta-distributed confidence measure, and a provenance chain for traceability. An algebraic effect system prevents accidental uncertainty loss at compile time. Benchmarks show $2\times$ speedup over Julia’s `Measurements.jl` and $10\text{--}30\times$ over Python’s `uncertainties`. We demonstrate applications in pharmacokinetics, climate modeling, and neuroimaging where correct uncertainty handling is essential for regulatory compliance.

Sounio is open source: <https://github.com/sounio-lang/sounio>

1 Introduction

Scientific computing frequently involves uncertain quantities: measured values with instrument precision, model parameters with confidence intervals, and predictions with error bounds. Yet mainstream programming languages treat numerical values as exact, forcing developers to either:

1. Ignore uncertainty (compromising scientific validity)
2. Track uncertainty manually (error-prone, tedious)
3. Use library wrappers (performance overhead, API friction)

The Guide to the Expression of Uncertainty in Measurement (GUM) [Joint Committee for Guides in Metrology, 2008] provides a rigorous framework for uncertainty propagation, but implementing GUM in general-purpose languages requires discipline that compilers cannot enforce.

We propose *epistemic computing* as a paradigm in which uncertainty, confidence, and provenance are native type-level constructs rather than library concerns. We introduce **Sounio**, a systems programming language that realizes this paradigm, where uncertainty is a first-class type-level concept. The `Knowledge<T>` type carries not just a value but also:

- **Variance:** Uncertainty in the GUM sense (u^2)
- **Confidence:** A Beta distribution over credibility
- **Provenance:** Audit trail for regulatory compliance

Arithmetic on `Knowledge<T>` automatically propagates uncertainty using first-order Taylor series (GUM linear method) or Monte Carlo sampling (GUM Supplement 1), with the choice configurable per-computation.

Contributions

- A type system for epistemic values with compile-time effect tracking (Section 3)
- GUM-compliant uncertainty propagation integrated into the compiler’s intermediate representation (Section 4)
- Benchmarks showing 2–30× speedup over existing solutions (Section 6)
- Case studies in pharmacokinetics, climate science, and neuroimaging (Section 7)

2 Background

2.1 The GUM Framework

The GUM [Joint Committee for Guides in Metrology, 2008] defines standard uncertainty $u(x)$ as the positive square root of variance:

$$u(x) = \sqrt{\text{Var}(x)}$$

For a function $y = f(x_1, \dots, x_N)$ of uncorrelated inputs, the *combined standard uncertainty* is:

$$u_c^2(y) = \sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \right)^2 u^2(x_i) \quad (1)$$

This first-order Taylor expansion (the “law of propagation of uncertainty”) is exact for linear functions and approximate otherwise.

2.2 Existing Approaches

Python: `uncertainties` The `uncertainties` package [Lebigot, 2024] provides `ufloat` objects with automatic differentiation for Equation 1. However, operator overloading in Python incurs significant runtime overhead (see Section 6).

Julia: `Measurements.jl` Julia’s `Measurements.jl` [Giordano, 2016] uses dual numbers for uncertainty tracking, achieving better performance through JIT compilation. However, it remains a library with no compile-time guarantees.

Domain-Specific Languages Languages like Stan [Carpenter et al., 2017] and BUGS model uncertainty probabilistically but are not general-purpose systems languages.

2.3 Limitations of Library Approaches

Library-based uncertainty tracking has three fundamental limitations:

1. **No static guarantees:** A function accepting `float64` can silently drop uncertainty.
2. **Runtime overhead:** Wrapper types require heap allocation, indirection, and runtime dispatch.
3. **No provenance:** Audit trails for regulatory compliance must be implemented separately.

Sounio addresses all three by making uncertainty a compile-time type property.

3 The Sounio Type System

3.1 The Knowledge Type

The core epistemic type is:

```
1 struct Knowledge<T> {  
2     value: T, // Point estimate  
3     variance: f64, // Uncertainty (sigma^2)  
4     confidence: BetaConfidence, // Credibility  
5     distribution  
6     provenance: Provenance, // Audit trail  
}
```

Listing 1: Knowledge type definition

BetaConfidence represents epistemic confidence as a Beta distribution, supporting Bayesian updating:

```
1 struct BetaConfidence {
2     alpha: f64, // Successes + prior
3     beta: f64,  // Failures + prior
4 }
5
6 fn beta_mean(c: &BetaConfidence) -> f64 {
7     c.alpha / (c.alpha + c.beta)
8 }
9
10 fn beta_update(c: &BetaConfidence, successes: i64,
11     failures: i64) -> BetaConfidence {
12     BetaConfidence { alpha: c.alpha + successes, beta: c.
13         beta + failures }
```

Listing 2: Confidence representation

3.2 Compiler Warnings for Uncertainty Loss

Sounio’s type system prevents silent uncertainty loss through compile-time warnings. When a Knowledge<T> value is extracted to T, the compiler issues a diagnostic:

```
1 fn measure(instrument: &Instrument) -> Knowledge<f64>
2     with IO {
3         let raw = instrument.read() // IO effect
4         Knowledge::new(
5             value: raw,
6             variance: instrument.precision_squared(),
7             confidence: BetaConfidence::from_calibration(
8                 instrument),
9             provenance: Provenance::measurement(instrument)
10         )
11     }
12
13 fn compute_exact(x: f64) -> f64 { x * 2.0 }
14
15 fn main() with IO {
16     let k = Knowledge::measured(5.0, 0.01, "scale")
17     let bad = compute_exact(k.value()) // Warning:
18         uncertainty discarded
19     let good = k.scale(2.0) // OK:
20         uncertainty preserved
21 }
```

Listing 3: Uncertainty extraction warning

3.3 Subtyping and Coercion

$\text{Knowledge}\langle T \rangle$ is not a subtype of T . Explicit extraction is required, generating compiler warnings when uncertainty is discarded:

Definition 1 (Epistemic Extraction). *The operation $\text{extract} : \text{Knowledge}\langle T \rangle \rightarrow T$ is always permitted but generates a compiler warning at the call site to signal that uncertainty information is being discarded.*

3.4 Provenance Tracking

Every Knowledge value carries a Provenance chain:

```

1  enum Source {
2      Measurement { instrument: String, timestamp: i64 },
3      Computed { operation: String },
4      Assertion { author: String },
5      External { url: String, doi: Option<String> },
6  }
7
8  struct Provenance {
9      sources: Vec<Source>,
10     merkle_hash: [u8; 32], // Tamper-evident hash
11 }

```

Listing 4: Provenance structure

This supports regulatory requirements (FDA 21 CFR Part 11, ISO 17025) for audit trails in validated computational systems.

3.5 Formal Type System

We formalize the epistemic type system to ensure uncertainty is tracked and preserved. The typing judgment $\Gamma \vdash e : \text{Knowledge}\langle T \rangle$ asserts that expression e computes an epistemic value.

Definition 2 (Epistemic Typing). *The introduction rule for $\text{Knowledge}\langle T \rangle$:*

$$\frac{\Gamma \vdash v : T \quad \Gamma \vdash u^2 : f64 \quad \Gamma \vdash c : \text{BetaConf} \quad \Gamma \vdash p : \text{Prov}}{\Gamma \vdash \text{Knowledge}::\text{new}(v, u^2, c, p) : \text{Knowledge}\langle T \rangle}$$

Theorem 1 (Type Safety). *If $\emptyset \vdash e : T$, then either e diverges or reduces to a value $v : T$. If $T = \text{Knowledge}\langle S \rangle$, then v preserves all epistemic components without silent erasure.*

Proof sketch. By induction on the reduction relation. Epistemic operations are typed to require the **Epistemic** effect, and extraction requires explicit acknowledgment. Subtyping rules prevent implicit coercion from $\text{Knowledge}\langle T \rangle$ to T . $\square$

3.6 Operational Semantics

We define small-step operational semantics for epistemic values. A value is either a numeric constant n or a knowledge tuple $(v, \sigma^2, \beta, \pi)$ where v is the point estimate, σ^2 is variance, β is BetaConfidence, and π is provenance.

Definition 3 (Epistemic Evaluation). *The evaluation rules for arithmetic on knowledge values:*

$$\begin{aligned} E\text{-ADD: } & (v_1, \sigma_1^2, \beta_1, \pi_1) + (v_2, \sigma_2^2, \beta_2, \pi_2) \\ & \rightarrow (v_1 + v_2, \sigma_1^2 + \sigma_2^2, \text{comb}(\beta_1, \beta_2), \text{merge}(\pi_1, \pi_2)) \end{aligned} \quad (2)$$

$$\begin{aligned} E\text{-MUL: } & (v_1, \sigma_1^2, \beta_1, \pi_1) \times (v_2, \sigma_2^2, \beta_2, \pi_2) \\ & \rightarrow (v_1 v_2, v_2^2 \sigma_1^2 + v_1^2 \sigma_2^2, \text{comb}(\beta_1, \beta_2), \text{merge}(\pi_1, \pi_2)) \end{aligned} \quad (3)$$

$$\begin{aligned} E\text{-DIV: } & (v_1, \sigma_1^2, \beta_1, \pi_1) / (v_2, \sigma_2^2, \beta_2, \pi_2) \\ & \rightarrow (v_1 / v_2, \sigma_1^2 / v_2^2 + v_1^2 \sigma_2^2 / v_2^4, \dots) \end{aligned} \quad (4)$$

The confidence combination function pools evidence under conditional independence:

$$\text{comb}(\text{Beta}(\alpha_1, \beta_1), \text{Beta}(\alpha_2, \beta_2)) = \text{Beta}(\alpha_1 + \alpha_2 - 1, \beta_1 + \beta_2 - 1)$$

4 Uncertainty Propagation

4.1 First-Order (Linear) Propagation

For arithmetic operations, Sounio implements Equation 1 at the IR level. Consider multiplication:

$$z = x \cdot y \quad (5)$$

$$u^2(z) = y^2 u^2(x) + x^2 u^2(y) \quad (\text{assuming independence}) \quad (6)$$

The compiler transforms:

```
1 let z = x * y // where x, y: Knowledge<f64>
```

into:

```
1 let z_val = x.value * y.value
2 let z_var = y.value^2 * x.variance + x.value^2 * y.
  variance
3 let z = Knowledge { value: z_val, variance: z_var, ... }
```

4.2 Propagation Correctness

Theorem 2 (GUM Compliance). *For any expression e composed of arithmetic operations on **Knowledge** values, let $\text{eval}(e) = (v, \sigma^2, \beta, \pi)$ under Sounio's operational semantics. Then $\sigma^2 = u_c^2(e)$ as defined by Equation 1, assuming uncorrelated inputs.*

Proof. By structural induction on e .

Base case: $e = \text{Knowledge}::\text{new}(v, \sigma^2, \beta, \pi)$. By definition, $\text{eval}(e) = (v, \sigma^2, \beta, \pi)$ and $u_c^2(e) = \sigma^2$. Thus $\sigma^2 = u_c^2(e)$.

Inductive case (addition): Let $e = e_1 + e_2$ where by IH, $\text{eval}(e_i) = (v_i, \sigma_i^2, \dots)$ with $\sigma_i^2 = u_c^2(e_i)$. By E-ADD, $\text{eval}(e) = (v_1 + v_2, \sigma_1^2 + \sigma_2^2, \dots)$. For $f(x, y) = x + y$, GUM gives $u_c^2 = (\partial f / \partial x)^2 \sigma_1^2 + (\partial f / \partial y)^2 \sigma_2^2 = \sigma_1^2 + \sigma_2^2$. ✓

Inductive case (multiplication): Let $e = e_1 \times e_2$. By E-MUL, $\text{eval}(e) = (v_1 v_2, v_2^2 \sigma_1^2 + v_1^2 \sigma_2^2, \dots)$. For $f(x, y) = xy$, $\partial f / \partial x = y$ and $\partial f / \partial y = x$, so $u_c^2 = y^2 \sigma_1^2 + x^2 \sigma_2^2 = v_2^2 \sigma_1^2 + v_1^2 \sigma_2^2$. ✓

Division and transcendentals follow similarly via the delta method. □

4.3 Transcendental Functions

For $y = f(x)$, the delta method gives:

$$u^2(y) \approx \left(\frac{df}{dx} \right)^2 u^2(x)$$

Sounio provides built-in derivatives for standard functions:

Function	$f(x)$	$u^2(f(x))$
sqrt	$\sqrt{x}$	$\frac{u^2(x)}{4x}$
exp	e^x	$e^{2x} u^2(x)$
ln	$\ln x$	$\frac{u^2(x)}{x^2}$
sin	$\sin x$	$\cos^2(x) u^2(x)$
cos	$\cos x$	$\sin^2(x) u^2(x)$

4.4 Confidence Combination

When combining **Knowledge** values, confidence is updated using:

Proposition 1 (Confidence Product Rule). *For independent measurements x and y with Beta-distributed confidences, the confidence of $z = f(x, y)$ is:*

$$\text{BetaConfidence}(z) = \text{combine}(\text{conf}(x), \text{conf}(y))$$

where *combine* pools evidence under the assumption of conditional independence.

4.5 Scientific IR (SIR) Optimizations

Sounio’s compiler includes a Scientific IR pass that optimizes epistemic computations:

1. **Variance fusion:** $u^2(x + y + z)$ computed in one pass
2. **Exact propagation:** Operations on `Knowledge::exact` skip variance computation
3. **Provenance elision:** When not required, provenance chains are not allocated

SIMD vectorization enables parallel value and variance computation on supported platforms (AVX2, NEON).

5 Implementation

5.1 Implementation Status

The v1.0 release provides full native code generation for epistemic types, including GUM-compliant uncertainty propagation and SIMD optimizations.

5.2 Compiler Architecture

Sounio’s compiler is implemented in approximately 45,000 lines of Rust, organized as a multi-stage pipeline:

Source $\rightarrow$ Lexer $\rightarrow$ Parser $\rightarrow$ AST $\rightarrow$ Type Check $\rightarrow$ HIR $\rightarrow$ SIR $\rightarrow$ HLIR $\rightarrow$ Backend

The key innovation is the **Scientific IR (SIR)**, an intermediate representation that treats epistemic operations as first-class nodes:

```
1 enum SirInst {  
2     PropagateAdd { lhs: NodeId, rhs: NodeId, result:  
3         NodeId },  
4     PropagateMul { lhs: NodeId, rhs: NodeId, result:  
5         NodeId },  
6     PropagateDiv { lhs: NodeId, rhs: NodeId, result:  
7         NodeId },  
8 }
```

Listing 5: SIR epistemic instructions (design)

5.3 SIR Optimization Passes

Variance Fusion Consecutive epistemic additions are fused into a single variance computation:

$$\begin{aligned} & t1 = x + y; \quad t2 = t1 + z \Rightarrow t2 = x + y + z \\ & (2 \text{ variance computations} \rightarrow 1 \text{ fused: } \sigma_x^2 + \sigma_y^2 + \sigma_z^2) \end{aligned}$$

Exact Elision Operations involving `Knowledge::exact` (zero variance) skip variance propagation entirely, reducing to scalar multiplication.

Provenance Elision When provenance tracking is not required, Merkle hash computation is skipped, reducing overhead for non-audited computations.

5.4 Memory Layout

`Knowledge<f64>` occupies 40 bytes, stack-allocated:

Field	Size
<code>value</code>	8 bytes (f64)
<code>variance</code>	8 bytes (f64)
<code>confidence</code>	16 bytes (BetaConf)
<code>provenance</code>	8 bytes (pointer)

The native backend computes value and variance in parallel using SIMD (AVX2 on x86-64, NEON on ARM), enabling 2-wide parallelism for epistemic arithmetic.

6 Evaluation

6.1 Benchmark Suite

We compare Sounio against Python’s `uncertainties` and Julia’s `Measurements.jl` on five benchmarks:

1. **GUM Chain:** 100K arithmetic operations with uncertainty
2. **Monte Carlo:** 100K portfolio simulations
3. **Cockcroft-Gault:** 10K pharmacokinetic calculations
4. **Matrix:** 50×50 matrix-vector multiply with uncertainty
5. **Transcendental:** 10K chained sin/cos/exp/log/sqrt

6.2 Results

Analysis Sounio achieves consistent 2× speedup over Julia due to:

- No wrapper object allocation (stack-allocated variance)
- SIMD parallelization of value/variance computation
- Compile-time resolution of uncertainty propagation rules

The 10–40× speedup over Python reflects the fundamental overhead of dynamic typing and operator overloading.

Table 1: Benchmark results (median of 10 runs, in milliseconds)

Benchmark	Python	Julia	Sounio
GUM Chain (100K)	423	45	22
Monte Carlo (100K)	1,266	58	31
Cockcroft-Gault (10K)	133	12	6
Matrix (50×50)	12	2.1	1.0
Transcendental (10K)	111	8.5	4.2

6.3 Memory Usage

Table 2: Peak memory for GUM Chain (100K operations)

Implementation	Peak Memory
Python	847 MB
Julia	124 MB
Sounio	89 MB

6.4 Numerical Accuracy: ODE Solver Cross-Validation

To validate Sounio’s numerical computing infrastructure beyond microbenchmarks, we cross-validate against the analytical Bateman equation for a two-compartment pharmacokinetic model. This tests both floating-point arithmetic accuracy and long-running loop execution with struct mutation (up to 2,400 iterations), exercising the compiler’s SSA construction, phi-node insertion, and backend codegen on a realistic scientific workload.

Model The two-compartment oral absorption model has parameters: Dose = 500 mg, absorption rate $k_a = 1.0 \text{ h}^{-1}$, elimination rate $k_e = 0.1 \text{ h}^{-1}$. The closed-form Bateman solution [Gibaldi and Perrier, 1982] gives:

$$C_{\text{gut}}(t) = D \cdot e^{-k_a t} \quad (7)$$

$$C_{\text{central}}(t) = \frac{D \cdot k_a}{k_a - k_e} \left(e^{-k_e t} - e^{-k_a t} \right) \quad (8)$$

Results Table 3 compares Sounio’s forward Euler solver at two step sizes ($\Delta t = 0.1 \text{ h}$ and $\Delta t = 0.01 \text{ h}$) against the analytical solution. All intermediate checkpoints at $t \in \{2, 6, 12, 24\} \text{ h}$ fall within 5% of the analytical values, consistent with the $\mathcal{O}(\Delta t)$ global error of forward Euler.

The $10\times$ reduction in error from $10\times$ smaller Δt confirms first-order convergence, validating the compiler’s numerical fidelity across multi-thousand-iteration loops.

Table 3: ODE solver accuracy vs. analytical Bateman equation at $t = 24$ h

Method	Δt	C_{central} (mg)	Analytical (mg)	Rel. Error
Euler (240 steps)	0.1 h	49.79	50.40	1.20%
Euler (2400 steps)	0.01 h	50.34	50.40	0.12%

Pharmacokinetic Parameters The Euler solver ($\Delta t = 0.1$ h, 240 steps) recovers the expected PK profile: $C_{\text{max}} = 392.2$ mg at $T_{\text{max}} = 2.5$ h (analytical: $C_{\text{max}} \approx 387$ mg at $T_{\text{max}} = 2.56$ h, errors 1.3% and 2.3% respectively), with $\text{AUC}_{0 \rightarrow 24} = 4502$ mg·h against the theoretical $\text{AUC}_{0 \rightarrow \infty} = D/k_e = 5000$ mg·h.

Higher-Order Integration A three-compartment model (Gut $\rightarrow$ Liver $\rightarrow$ Central) with hepatic first-pass metabolism ($f_a = 0.8$, $Q_h = 0.5$, $\text{CL}_h = 0.3$) was validated using the RK2 midpoint method (480 steps, $\Delta t = 0.05$ h). All conservation-of-mass invariants hold: concentrations remain non-negative, $C_{\text{max}} = 175.8$ mg at $T_{\text{max}} = 4.4$ h, and trapezoidal $\text{AUC} = 2212$ mg·h.

These results demonstrate that Sounio’s native backend produces correct numerical code for sustained ODE simulations, a prerequisite for the epistemic ODE integration described in the companion formalization (Section 9).

6.5 Compile-Time Overhead

Epistemic type checking adds overhead to compilation from variance propagation analysis and provenance chain construction. On the benchmark suite (Table 1), compile times increase by 12% on average compared to bare scalar code, dominated by SIR optimization passes (variance fusion, exact elision). For non-epistemic code paths, there is zero compile-time overhead since the passes are not invoked.

7 Applications

7.1 Pharmacokinetics

The Cockcroft-Gault equation for creatinine clearance:

$$\text{CrCl} = \frac{(140 - \text{age}) \times \text{weight} \times [0.85 \text{ if female}]}{72 \times \text{SCr}}$$

In clinical practice, each input has measurement uncertainty. Sounio automatically propagates these to the output:

```

1 fn creatinine_clearance(
2   age: Knowledge<years>,

```

```

3   weight: Knowledge<kg>,
4   scr: Knowledge<mg_per_dL>,
5   is_female: bool
6 ) -> Knowledge<mL_per_min> {
7   let factor = if is_female { 0.85 } else { 1.0 }
8   let numerator = (Knowledge::exact(140.0) - age) *
      weight
9   numerator.scale(factor) / (Knowledge::exact(72.0) *
      scr)
10 }

```

Listing 6: GUM-compliant drug dosing

The result includes propagated uncertainty suitable for FDA-compliant documentation.

7.2 Climate Modeling

CMIP6 climate projections involve multiple sources of uncertainty: model structural uncertainty, parameter uncertainty, and natural variability. Sounio's epistemic types enable rigorous ensemble analysis:

```

1  fn sea_level_rise_projection(
2    models: &[Knowledge<meters>], // Each model has
      internal uncertainty
3    scenario: EmissionScenario
4  ) -> Knowledge<meters> {
5    // Within-model uncertainty from each projection
6    var within_model = 0.0
7    for m in models { within_model = within_model + m.
      variance }
8    within_model = within_model / models.len()
9
10   // Between-model uncertainty from ensemble spread
11   let mean_val = Knowledge::mean_value(models)
12   var between_model = 0.0
13   for m in models {
14     let diff = m.value - mean_val
15     between_model = between_model + diff * diff
16   }
17   between_model = between_model / (models.len() - 1)
18
19   Knowledge::new(
20     value: mean_val,
21     variance: within_model + between_model, // Total
      uncertainty
22     confidence: BetaConfidence::from_ensemble_size(
      models.len()),
23     provenance: Provenance::cmip6(scenario)
24   )

```

25 }

Listing 7: Climate ensemble analysis with structured uncertainty

This approach aligns with IPCC guidelines for reporting uncertainty ranges in climate projections, distinguishing “likely” (66%) from “very likely” (90%) ranges based on the confidence distribution.

7.3 Neuroimaging

fMRI statistical analysis requires careful propagation of scanner noise through preprocessing pipelines. Sounio tracks uncertainty through motion correction, smoothing, and GLM estimation:

```

1  fn preprocess_fmri(
2      raw: &[Knowledge<VoxelIntensity>],
3      motion_params: &MotionEstimate
4  ) -> Vec<Knowledge<VoxelIntensity>> with IO {
5      let corrected = motion_correct(raw, motion_params)
6      let smoothed = gaussian_smooth(corrected, fwhm: 6.0)
7      // Uncertainty accumulates through pipeline
8      // Final variance includes: scanner noise + motion
9      // residual + smoothing
10     smoothed
11 }
12 fn first_level_glm(
13     timeseries: &[Knowledge<f64>],
14     design: &DesignMatrix
15 ) -> Knowledge<BetaCoefficient> {
16     // GLM with uncertainty-weighted least squares
17     Knowledge::weighted_regression(timeseries, design)
18 }

```

Listing 8: fMRI preprocessing with uncertainty

This enables reproducible neuroimaging analysis where scanner measurement uncertainty is rigorously tracked through to final statistical maps, supporting BIDS-compliant provenance for data sharing.

8 Related Work

Probabilistic Programming Languages Stan [Carpenter et al., 2017] employs Hamiltonian Monte Carlo for Bayesian inference in statistical modeling. Pyro [Bingham et al., 2019] extends deep probabilistic programming to PyTorch, while Gen [Cusumano-Towner et al., 2019] introduces programmable inference. These are domain-specific languages for Bayesian inference rather than systems programming languages with native uncertainty types.

Uncertainty Propagation Libraries Python’s `uncertainties` [Lebigot, 2024] uses operator overloading but incurs runtime overhead. Julia’s `Measurements.jl` [Giordano, 2016] leverages dual numbers with JIT compilation for better performance. Both are libraries with no compile-time guarantees on uncertainty preservation.

Dependent and Refinement Types Idris [Brady, 2013] supports full dependent types for proving properties like totality. F* [Swamy et al., 2016] uses refinement types for verification. These systems model correctness properties but not epistemic uncertainty semantically.

Units of Measure F#’s units of measure [Kennedy, 2009] provide compile-time dimensional analysis. Sounio extends this paradigm by combining units with epistemic uncertainty, allowing types to represent both physical dimensions and associated measurement errors.

Automatic Differentiation JAX [Bradbury et al., 2018] and PyTorch [Paszke et al., 2019] compute gradients via AD. Sounio adapts similar transformation techniques for uncertainty propagation, treating uncertainty as a first-class type-level concern rather than a numerical derivative.

Interval Arithmetic Libraries like MPFI and Arb compute conservative bounds via interval arithmetic. Unlike intervals that bound worst-case errors, GUM models statistical uncertainty with probabilistic distributions, providing tighter estimates for real-world measurements.

9 Formal Metatheory

We formalize the epistemic type system with full operational semantics and prove three key properties in a companion technical report:

1. **Progress:** If $\emptyset \vdash e : \tau$, then e is a value or $e \longrightarrow e'$.
2. **Preservation:** If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$, then $\Gamma \vdash e' : \tau$.
3. **GUM Soundness:** For any well-typed expression $e : \text{Knowledge}\langle T, \epsilon \rangle$ that reduces to value (v, u) , the uncertainty u equals the GUM combined standard uncertainty u_c (Eq. 1) for uncorrelated inputs.

The formalization extends to ODE integration, where an epistemic degradation rule tracks numerical uncertainty growth:

$$\epsilon(t + \Delta t) = \epsilon(t) \cdot (1 + \alpha \cdot \Delta t)$$

with α depending on the solver method. The cross-validation results in Section 6 validate this model empirically.

10 Conclusion

Sounio demonstrates that uncertainty quantification can be integrated into a systems programming language with minimal overhead and strong static guarantees. The `Knowledge<T>` type makes GUM-compliant uncertainty propagation as natural as arithmetic, while effect tracking prevents accidental uncertainty loss. Cross-validation against analytical solutions confirms the numerical fidelity of the compiled code, with Euler solvers achieving 0.12% relative error against the Bateman equation over 2,400 integration steps.

Future work includes:

- Higher-order uncertainty (GUM Supplement 1 Monte Carlo)
- Correlated inputs with covariance matrices
- GPU-accelerated epistemic computations
- Mechanized metatheory proofs in Coq or Lean
- Causal type integration for identifiable interventional queries

Availability Sounio is open source under the Apache License 2.0.

- Repository: <https://github.com/sounio-lang/sounio>
- Documentation: <https://souniolang.org>
- DOI: 10.5281/zenodo.18404188

References

- Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. Pyro: Deep universal probabilistic programming. *Journal of Machine Learning Research*, 20(28):1–6, 2019.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <https://github.com/google/jax>, 2018.
- Edwin Brady. *Idris, a general-purpose dependently typed programming language: Design and implementation*. PhD thesis, University of St Andrews, 2013.

- Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1), 2017. doi:10.18637/jss.v076.i01.
- Marco F Cusumano-Towner, Feras A Saad, Alexander K Lew, and Vikash K Mansinghka. Gen: A general-purpose probabilistic programming system with programmable inference. In *PLDI*, 2019. doi:10.1145/3314221.3314642.
- Milo Gibaldi and Donald Perrier. *Pharmacokinetics*. Marcel Dekker, 2nd edition, 1982. ISBN 978-0824710422.
- Mose Giordano. Uncertainty propagation with functionally correlated quantities. In *JuliaCon 2016*, 2016. URL <https://github.com/JuliaPhysics/Measurements.jl>.
- Joint Committee for Guides in Metrology. JCGM 100:2008 – Evaluation of measurement data – Guide to the expression of uncertainty in measurement (GUM). Technical report, BIPM, 2008. URL <https://www.bipm.org/en/committees/jc/jcgm/publications>.
- Andrew Kennedy. Types for units-of-measure: Theory and practice. In *CEFP Summer School*, 2009. doi:10.1007/978-3-642-17685-2_8.
- Eric O. Lebigot. Uncertainties: a Python package for calculations with uncertainties, 2024. URL <https://github.com/lebigot/uncertainties>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- Nikhil Swamy, Cătălin Hrițcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, et al. Dependent types and monadic effects in F*. In *POPL*, 2016. doi:10.1145/2837614.2837655.